

Mixed-Integer Programming Formulation

Unrelated Parallel Machine Scheduling with Sequence-Dependent Setup Times

Josa Ferreira*

June 18, 2026

Abstract

This portfolio project presents two Mixed-Integer Linear Programming (MILP) models for the unrelated parallel machine scheduling problem with sequence-dependent setup times, $R \mid s_{ij} \mid C_{\max}$ and $R \mid s_{ij} \mid \sum_j (E_j + T_j)$. The first formulation minimizes the total makespan ($C_{\max}$) for independent jobs across heterogeneous machines, while the second minimizes the sum of earliness and tardiness of jobs. Featuring a complete mathematical framework including assignment, flow conservation, and subtour elimination constraints, this project serves as a technical demonstration of applying mathematical optimization to complex manufacturing and logistics challenges.

Contents

1 Mathematical Formulations	1
1.1 Makespan Problem	1
1.2 Earliness and Tardiness Problem	2
2 Computational Results	4

Keywords: Parallel Machine Scheduling Problem; Makespan; Earliness and Tardiness; Sequence Dependent Setup.

1 Mathematical Formulations

Both models are presented in this section. Both models are presented in this section. Even though the earliness and tardiness problem includes all sets, parameters, and variables, and some constraints of the makespan problem, they will be reintroduced for the sake of clarity and completeness.

1.1 Makespan Problem

1.1.1 Problem Description

The problem $R \mid s_{ij} \mid C_{\max}$ consists of scheduling a set of independent jobs on a set of unrelated parallel machines. Under this configuration, each job must be processed exactly once on any machine. Because the machines are unrelated, they process jobs at varying speeds. Furthermore, transitioning between consecutive jobs on the same machine incurs a sequence-dependent setup time. The objective is to minimize the makespan, defined as the time at which the last job completes processing.

1.1.2 Sets

- $M = \{1, 2, \dots, m\}$: Set of m unrelated parallel machines.
- $N = \{1, 2, \dots, n\}$: Set of n available jobs.
- $N_0 = N \cup \{0\}$: Extended set of jobs, including a dummy source node (0) representing the start of a machine's sequence.

*josenildo.junior9321@gmail.com

1.1.3 Parameters

- p_{ij} : Processing time of job $j \in N$ on machine $i \in M$.
- s_{ijk} : Sequence-dependent setup time incurred when transitioning from job $j \in N_0$ to job $k \in N_0$ on machine $i \in M$.

1.1.4 Decision Variables

- $x_{ijk} = 1$ if job $j \in N_0$ immediately precedes job $k \in N_0$ on machine $i \in M$; 0 otherwise.
- $y_{ij} = 1$ if job $j \in N$ is processed on machine $i \in M$; 0 otherwise.
- $C_{\max}$: Continuous variable representing the overall makespan.

1.1.5 Model

The mixed-integer linear programming (MILP) model is formulated as follows:

$$\min C_{\max} \quad (1)$$

$$\text{S.t.} \quad \sum_{i \in M} y_{ij} = 1 \quad \forall j \in N \quad (2)$$

$$\sum_{\substack{k \in N_0 \\ k \neq j}} x_{ijk} = y_{ij} \quad \forall i \in M, j \in N \quad (3)$$

$$\sum_{\substack{j \in N_0 \\ j \neq k}} x_{ijk} = \sum_{j \in N_0, j \neq k} x_{ikj} \quad \forall i \in M, k \in N \quad (4)$$

$$\sum_{k \in N_0} x_{i0k} = 1 \quad \forall i \in M \quad (5)$$

$$\sum_{j \in S} \sum_{k \in N_0 \setminus S} x_{ijk} \geq y_{ih}, \quad \forall h \in S, S \subset N_0, i \in M \quad (6)$$

$$\sum_{j \in N_0} \sum_{\substack{k \in N_0 \\ k \neq j}} s_{ijk} x_{ijk} + \sum_{j \in N} p_{ij} y_{ij} \leq C_{\max}, \quad \forall i \in M \quad (7)$$

The objective function (1) minimizes the makespan ($C_{\max}$). Constraints (2) ensure that each job is assigned to and processed by exactly one machine. Constraints (3) link the routing and assignment variables, dictating that if a job is processed on a machine, it must immediately precede another node in that machine's sequence. Constraints (4) enforce flow conservation, guaranteeing that a machine must leave any job node it enters. Constraints (5) require every machine to initiate its routing sequence from the dummy origin (node 0). Constraints (6) represent the subtour elimination constraints (SECs), which prevent invalid isolated loops within the job sequences. Subtour elimination constraints were implemented as lazy constraints, given that their total count grows exponentially with the number of jobs. Finally, Constraints (7) defines makespan.

1.2 Earliness and Tardiness Problem

1.2.1 Problem Description

The problem $R \mid s_{ij} \mid \sum_j (E_j + T_j)$ consists of scheduling a set of independent jobs on a set of unrelated parallel machines. Each job must be processed exactly once on one of the machines and has an associated due date by which it should ideally be completed. Because the machines are unrelated, processing speeds vary from machine to machine for the same job. Furthermore, transitioning between consecutive jobs on the same machine incurs a sequence-dependent setup time. The objective is to minimize the total earliness and tardiness across all jobs, where earliness and tardiness are measured as the difference between a job's completion time and its due date.

1.2.2 Sets

- $M = \{1, 2, \dots, m\}$: Set of m unrelated parallel machines.
- $N = \{1, 2, \dots, n\}$: Set of n available jobs.
- $N_0 = N \cup \{0\}$: Extended set of jobs, including a dummy source node (0) representing the start of a machine's sequence.

1.2.3 Parameters

- p_{ij} : Processing time of job $j \in N$ on machine $i \in M$.
- s_{ijk} : Sequence-dependent setup time incurred when transitioning from job $j \in N_0$ to job $k \in N_0$ on machine $i \in M$.
- d_j : Due date of job $j \in N$.

1.2.4 Decision Variables

- $x_{ijk} = 1$ if job $j \in N_0$ immediately precedes job $k \in N_0$ on machine $i \in M$; 0 otherwise.
- $y_{ij} = 1$ if job $j \in N$ is processed on machine $i \in M$; 0 otherwise.
- C_j : Continuous variable representing the completion time of job $j \in N_0$.
- E_j : Continuous variable representing the earliness of job $j \in N$.
- T_j : Continuous variable representing the tardiness of job $j \in N$.

1.2.5 Model

The mixed-integer linear programming (MILP) model is formulated as follows:

$$\min \sum_{j \in N} (E_j + T_j) \quad (8)$$

$$\text{S.t.} \quad \sum_{i \in M} y_{ij} = 1 \quad \forall j \in N \quad (9)$$

$$\sum_{\substack{k \in N_0 \\ k \neq j}} x_{ijk} = y_{ij} \quad \forall i \in M, j \in N \quad (10)$$

$$\sum_{\substack{j \in N_0 \\ j \neq k}} x_{ijk} = \sum_{j \in N_0, j \neq k} x_{ikj} \quad \forall i \in M, k \in N \quad (11)$$

$$\sum_{k \in N_0} x_{i0k} = 1 \quad \forall i \in M \quad (12)$$

$$C_k \geq C_j + s_{ijk} + p_{ik} - M(1 - x_{ijk}) \quad \forall i \in M, j \in N_0, k \in N, j \neq k \quad (13)$$

$$C_j = d_j + T_j - E_j \quad \forall j \in N \quad (14)$$

The objective function (8) minimizes the sum of all earliness and tardiness of all jobs. Constraints (9) ensure that each job is assigned to and processed by exactly one machine. Constraints (10) link the routing and assignment variables, dictating that if a job is processed on a machine, it must immediately precede another node in that machine's sequence. Constraints (11) enforce flow conservation, guaranteeing that a machine must leave any job node it enters. Constraints (12) require every machine to initiate its routing sequence from the dummy origin (node 0). Constraints (13) compute the completion time of a job, depending on which job immediately precedes it and on which machine the processing occurs. Finally, Constraints (14) define the earliness or tardiness of a job.

2 Computational Results

Following the benchmark protocol proposed by [1] for $R \mid s_{ij} \mid C_{\max}$, 20 instances were generated: 10 with $m = 2$ machines and $n = 5$ jobs, and 10 with $m = 4$ machines and $n = 10$ jobs. Processing times (p) and setup times (s) were independently sampled from the discrete uniform distribution $UD(1, 99)$. For the earliness and tardiness model, due dates (d_j) were additionally generated for each job j following $UD(\max_{i \in M}(p_{ij}) + \max_{i \in M}(s_{i0j}), \frac{1.5n}{m}(\bar{p} + \bar{s}))$, where $\bar{p}$ and $\bar{s}$ corresponds to mean processing time and mean setup time, respectively. To ensure reproducibility, all generated instances and their corresponding results were stored in a SQLite database (`instances.sqlite`).

Table 1 presents the optimal makespan ($C_{\max}$) and solver runtime (in seconds) obtained for the 20 randomly generated instances under the two problem configurations.

Table 1: Computational results for the makespan model.

(a) $m = 2, n = 5$					(b) $m = 4, n = 10$				
m	n	Instance	$C_{\max}$	Time (s)	m	n	Instance	$C_{\max}$	Time (s)
2	5	0	152.0	0.091	4	10	0	171.0	5.279
2	5	1	134.0	0.034	4	10	1	168.0	18.464
2	5	2	165.0	0.084	4	10	2	174.0	21.147
2	5	3	172.0	0.044	4	10	3	172.0	3.170
2	5	4	176.0	0.063	4	10	4	146.0	11.347
2	5	5	139.0	0.056	4	10	5	129.0	11.859
2	5	6	252.0	0.193	4	10	6	167.0	10.793
2	5	7	188.0	0.045	4	10	7	148.0	4.912
2	5	8	172.0	0.021	4	10	8	140.0	4.547
2	5	9	169.0	0.077	4	10	9	149.0	3.899
Mean			171.9	0.071	Mean			156.4	9.542

The results illustrate the expected growth in computational effort as instance size increases. For $m = 2$, $n = 5$, the solver consistently finds the optimum in under 0.2 seconds (average ≈ 0.071 s). Scaling to $m = 4$, $n = 10$ raises the average runtime to ≈ 9.54 s, with the hardest instance (index 2) requiring 21.147 seconds.

Table 2 presents the analogous results for the earliness and tardiness model, using the objective value $\sum_j (E_j + T_j)$ in place of $C_{\max}$.

Table 2: Computational results for the earliness and tardiness model.

(a) $m = 2, n = 5$					(b) $m = 4, n = 10$				
m	n	Instance	$\sum_j (E_j + T_j)$	Time (s)	m	n	Instance	$\sum_j (E_j + T_j)$	Time (s)
2	5	0	79.0	0.166	4	10	0	0.0	1.959
2	5	1	48.0	0.158	4	10	1	23.0	2.186
2	5	2	7.0	0.084	4	10	2	9.0	1.801
2	5	3	72.0	0.133	4	10	3	23.0	0.429
2	5	4	0.0	0.023	4	10	4	4.0	4.361
2	5	5	0.0	0.021	4	10	5	39.0	7.892
2	5	6	2.0	0.091	4	10	6	0.0	2.959
2	5	7	56.0	0.105	4	10	7	0.0	1.712
2	5	8	9.0	0.052	4	10	8	5.0	4.229
2	5	9	6.0	0.048	4	10	9	12.0	2.386
Mean			27.9	0.088	Mean			11.5	2.991

Compared to the makespan model, the earliness and tardiness model solves substantially faster across

both instance sizes, with average runtimes of ≈ 0.088 s for $m = 2, n = 5$ and ≈ 2.99 s for $m = 4, n = 10$. Several instances achieve an objective value of zero, indicating that all jobs were completed exactly on their due dates; this is expected since due dates were generated with enough slack to be frequently attainable.

References

- [1] Eva Vallada and Rubén Ruiz. A genetic algorithm for the unrelated parallel machine scheduling problem with sequence dependent setup times. *European Journal of Operational Research*, 211(3):612–622, 2011.